eUprava: Mikroservis Saobraćajne Policije

Vukašin Čađenović

Fakultet tehničkih nauka

Univerzitet u Novom Sadu

Trg Dositeja Obradovića 6, 21000 Novi Sad

cadjenovic.sr34.2021@uns.ac.rs

Sažetak—*Proces evidentiranja saobraćajnih prekršaja, plaćanja kazni, kao i prijava ukradenih vozila tradicionalno zahtevaju fizičku prisutnost građana u policijskim stanicama Ministarstva unutrašnjih poslova (MUP), što rezultuje gužvama i dugim čekanjem. Digitalizacija ovog procesa kroz platformu eUprava omogućila bi građanima i službenicima transparentan, efikasan i digitalizovan način upravljanja ovim entitetima. Ovaj rad predstavlja razvoj nezavisnog mikroservisa TrafficPoliceService koji simulira rad i evidenciju Saobraćajne policije u okviru većeg e-government sistema za registraciju vozila. Sistem omogućava policijskim službenicima izdavanje saobraćajnih prekršaja, evidentiranje saobraćajnih nezgoda, pregled dosijea vozila, i upravljanje aktivnim nalozima (markiranje vozila). Građanima je omogućen uvid u sopstvene prekršaje, istoriju nezgoda i prijava ukradenih vozila. Omogućeno je i elektronsko plaćanje kazni uz digitalni potpis i obavezno dostavljanje dokaza o uplati. Poseban naglasak stavljen je na integraciju sa VehicleService i NotificationService mikroservisima, gde status vozila direktno utiče na mogućnost njegove registracije, a sistem automatski šalje relevantna obaveštenja. Celokupan sistem je nezavisan, pakovan u kontejnere što omogućava lako pokretanje i razmeštanje.*

Ključne reči—*saobraćajna policija; eUprava; digitalizacija javnih usluga; mikroservisi; distribuirani sistemi; Golang; Angular*

1. Uvod

Tradicionalni procesi komunikacije sa Saobraćajnom policijom u Republici Srbiji često podrazumevaju lični dolazak na šaltere, bilo da je u pitanju plaćanje saobraćajnih kazni, provera statusa vozila ili prijava krađe. Građani moraju doneti dokaze o uplati, čekati u redovima, dok je sa druge strane i sam proces evidentiranja saobraćajnih prekršaja od strane službenika fragmentisan i oslanja se na manuelnu obradu podataka. U kontekstu sve veće digitalizacije javnih usluga, ovaj proces predstavlja prostor za značajno unapređenje.

Razvojem *TrafficPolice* mikroservisa u okviru šire platforme eUprave, građanima se omogućava transparentan uvid u njihov dosije – od neplaćenih kazni do statusa prijavljenih ukradenih vozila, sve kroz jedinstvenu tačku pristupa. S druge strane, policijskim službenicima i administratorima sistem pruža moćan i brz alat za evidentiranje saobraćajnih incidenata, izdavanje kazni, pregled detaljnih informacija o vozilu i vlasniku, kao i markiranje sumnjivih vozila.

Najveća snaga ovog servisa leži u njegovoj integraciji sa ostalim mikroservisima unutar eUprava ekosistema, prvenstveno *VehicleService*-om. Pre nego što građanin može da registruje vozilo ili

izvrši prenos vlasništva, glavni servis komunicira sa *TrafficPolice* servisu kako bi proverio da li je vozilo ukradeno, pod poternicom ili ima neizmirene kazne. Time se stvara bezbedan i međusobno uslovljen sistem koji replicira prave provere unutar državnih institucija.

2. Srodna istraživanja

2.1. Elektronske usluge u javnoj upravi

Digitalizacija javnih servisa je globalni trend koji ubrzava efikasnost državne uprave i smanjuje troškove. Estonija je pionir u implementaciji e-government sistema sa 99% javnih usluga dostupnih online [2]. Njihov pristup baziran na X-Road platformi omogućava sigurnu razmenu podataka između različitih državnih institucija kroz standardizovane API interfejse, što je konceptualno slično mikroservisnoj arhitekturi primenjenoj u ovom projektu.

Sistem e-Uprava u Hrvatskoj [3] takođe omogućava građanima pristup različitim uslugama kroz jedinstvenu tačku pristupa, koristeći nacionalni autentifikacioni sistem (NIAS). Njihovo iskustvo pokazuje da centralizovana autentifikacija smanjuje administrativno opterećenje i povećava zadovoljstvo korisnika. Na sličan način, ovaj projekat koristi centralizovan *AuthService* koji izdaje JWT tokene, omogućavajući “Single Sign-On” (SSO) pristup svim servisima.

2.2. Mikroservisne arhitekture u državnim sistemima

Mikroservisi predstavljaju arhitekturni stil koji strukturira aplikaciju kao kolekciju malih, nezavisnih servisa organizovanih oko poslovnih mogućnosti [4]. Svaki servis je samostalno razmeštiva komponenta koja komunicira sa drugim servisima kroz lake mehanizme, najčešće HTTP REST API.

Dok su privatne kompanije prve usvojile mikroservisnu arhitekturu, njena primena u državnim institucijama donosi specifične prednosti: - **Nezavisnost timova:** Različiti timovi mogu raditi na različitim servisima u različitim tehnologijama (npr. *TrafficPoliceService* je pisan u Go-u, dok je *VehicleService* u C#/.NET-u). - **Otpornost na greške:** Pad jednog servisa ne dovodi do pada celog sistema. Ukoliko *NotificationService* padne, Saobraćajna policija i dalje može da beleži prekršaje. - **Tehnološka sloboda i skalabilnost:** Mogućnost skaliranja specifičnih servisa (npr. provera dosijea) zavisno od opterećenja.

Međutim, mikroservisi donose i izazove, primarno vezane za kompleksnost distribuiranih sistema i održavanje konzistentnosti podataka između servisa.

3. Specifikacija zahteva

3.1. Projektni zahtevi

Zahtevi za izradu projekta postavljeni od strane profesora su sledeći:

Arhitekturni zahtevi: - Svaki član tima mora da ima poseban mikroservis - Docker kontejnerizacija je obavezna za sve servise - Sistem mora imati centralizovanu autentifikaciju sa Single Sign-On (SSO) metodom

Komunikacija i integracija: - Servisi moraju koristiti REST API za međusobnu komunikaciju - Neophodno je implementirati minimum 3 razmene podataka između servisa

Funkcionalnost: - Svaki servis mora imati minimum 5 funkcionalnosti izvan osnovnih CRUD operacija - Svaki servis mora imati sopstvenu bazu podataka

Kontrola verzija: - Git repozitorijum mora biti javan - Celokupan kod mora biti na main grani pre odbrane projekta

4. Specifikacija dizajna

4.1. Arhitektura Sistema

Sistem je implementiran kao mikroservisna arhitektura sa četiri nezavisna servisa koja se izvršavaju unutar Docker kontejnera. *TrafficPoliceService* komunicira sa *AuthService*-om (za validaciju JWT tokena), *VehicleService*-om (za dobijanje detaljnih informacija o vozilu), i *NotificationService*-om (za slanje email obaveštenja prilikom izdavanja kazni).

Pregled servisa:

1. **AuthService (.NET)**
   - Upravljanje korisnicima (registracija, login, profil)
   - Generisanje i validacija JWT tokena
   - Upravljanje ulogama (User/Admin/TrafficOfficer)
2. **VehicleService (.NET)**
   - Upravljanje vozilima i registracijama
   - Poseduje detaljne informacije o vozilu (marka, model, godina, vlasnik)
   - Komunicira sa *TrafficPoliceService*-om pre odobravanja registracije.
3. **NotificationService (.NET)**
   - Slanje email obaveštenja
   - Asinhrona komunikacija za obaveštavanje korisnika (npr. kada se izda nova kazna).
4. **TrafficPoliceService (Golang)**
   - Izdavanje i upravljanje saobraćajnim prekršajima (Violations).
   - Evidencija saobraćajnih nezgoda (Accidents).
   - Upravljanje bazom ukradenih vozila (Stolen Vehicles).
   - Upravljanje nalozima/markiranjem vozila (Warrants/Flags).

- Provera kompletnog dosijea vozila (Dossier) uz dobavljanje dodatnih metapodataka o vozilu iz *VehicleService*-a.
- Poseduje sopstvenu SQL Server bazu podataka (*TrafficPoliceDb*).

4.2. Arhitektura TrafficPolice Servisa

Odluka da se *TrafficPoliceService* razvije u programskom jeziku **Go (Golang)** proizašla je iz potrebe za visokim performansama, malim utroškom memorije i odličnom podrškom za konkurentno programiranje.

Struktura servisa obuhvata: - **Gin Web Framework:** Korišćen za efikasno rutiranje HTTP zahteva i definisanje REST API-ja. Omogućava brzu obradu zahteva. - **GORM:** Upotrebljen za rad sa *Microsoft SQL Server* bazom podataka, omogućavajući automatizovanu migraciju šema i rad sa objektima (Officer, Violation, Accident, StolenVehicle, VehicleFlag). - **JWT Middleware:** Obezbeđuje sigurnost (RBAC). Middleware presreće zahteve i dekodira JWT token proveravajući pravo pristupa na osnovu uloge (TrafficOfficer za kreiranje prekršaja). - **HTTP Proxy/Klijent:** Kreiran interni modul unutar Go servisa koji komunicira sa *VehicleService* API-jem kako bi preuzeo tačne podatke o marki i modelu vozila bez dupliranja tih podataka u sopstvenoj bazi.

5. Implementacija i Funkcionalnosti

Korišćene tehnologije:

Backend: - Go (Golang) 1.23 za izradu API-ja *TrafficPoliceService* [1] - Gin Web Framework za rutiranje [2] - GORM za upravljanje bazom podataka [3] - gofpdf za generisanje PDF dokumenata na serveru

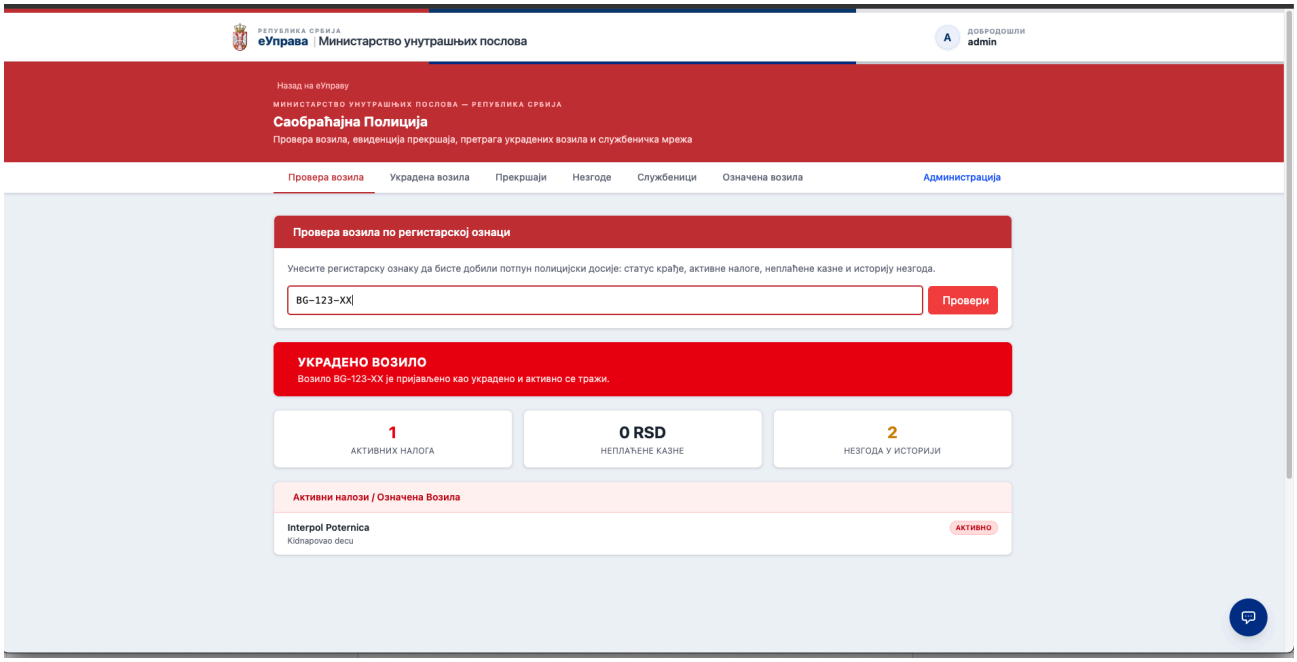
Baza podataka: - Microsoft SQL Server 2022 [9]

Frontend: - Angular 20 (Standalone komponente) za razvoj klijentske aplikacije [10] - Tailwind CSS v4 za stilizovanje [11] - PrimeNG biblioteka za UI komponente [6] - TypeScript 5.6 za tipiziran razvoj i lakše održavanje [12]

DevOps: - Docker za kontejnerizaciju aplikacija [13] - Docker Compose za orkestraciju mikroservisa

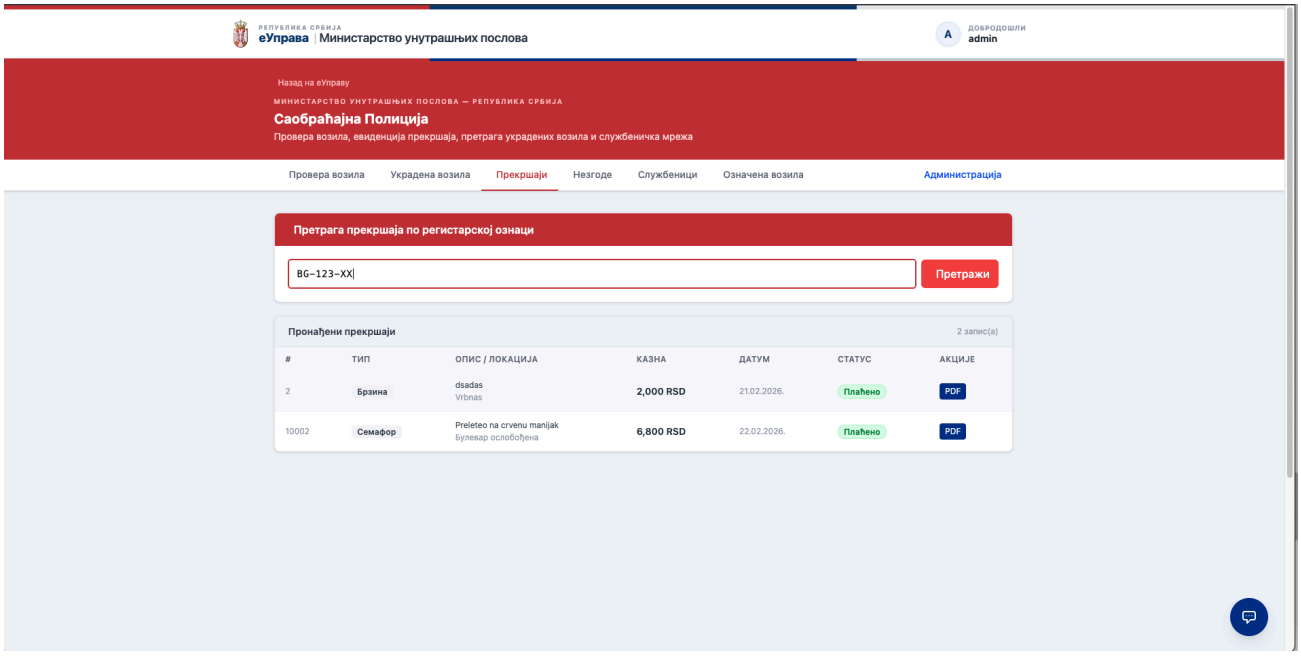
5.1. Agregacija Podataka (Vehicle Dossier)

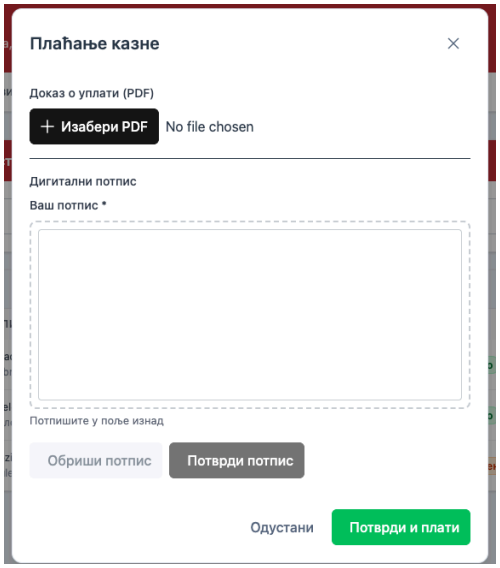
Najmoćniji endpoint ovog servisa je onaj za dobavljanje kompletnog dosijea vozila. Policijski službenici unose registarsku oznaku i dobijaju pregled svih relevantnih podataka. Bekend komunicira sa *VehicleService*-om da dohvati metapodatke (model, marka), a zatim iz sopstvene baze agregira: - Neplaćene prekršaje i ukupan dug - Istoriju učešća u nezgodama - Aktivne poternice/markiranja - Status da li je vozilo prijavljeno kao ukradeno



5.2. Upravljanje Prekršajima i Plaćanje

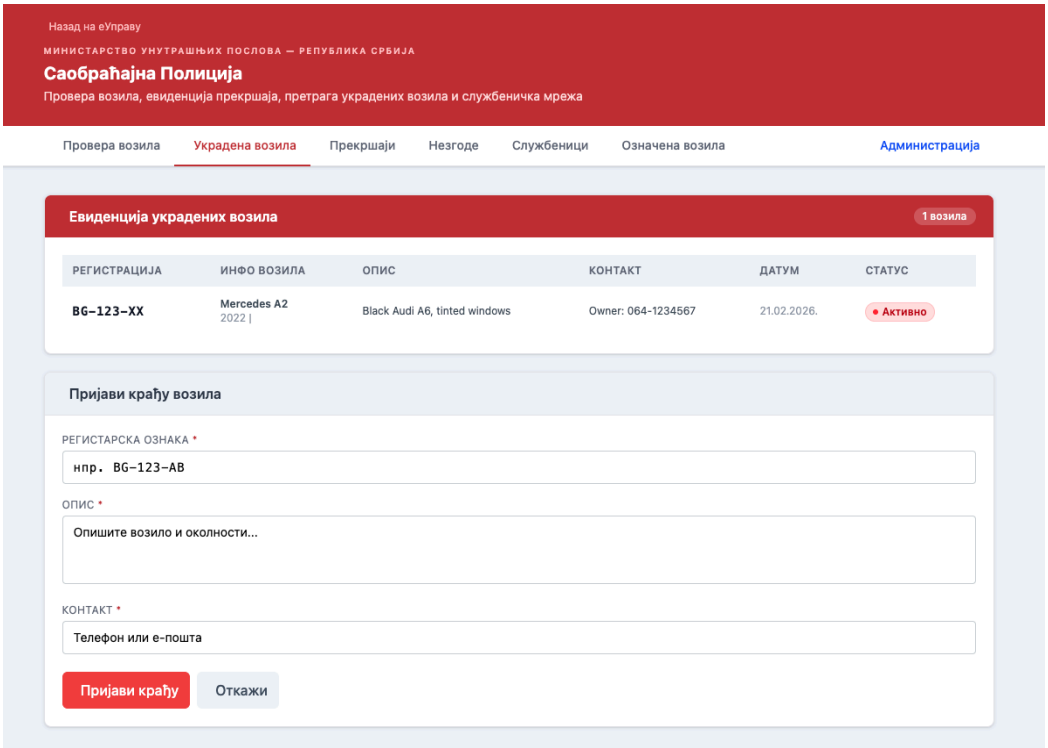
Službenici MUP-a putem zaštićenog Admin panela izdaju prekršaje koji se vezuju za registarsku tablicu. U trenutku izdavanja kazne, *TrafficPoliceService* upućuje zahtev *NotificationService*-u koji vozaču šalje email obaveštenje o prekršaju. Građani imaju mogućnost da kroz interfejs pregledaju svoje neplaćene prekršaje. Implementirana je funkcionalnost **plaćanja kazni** koja zahteva: 1. Otpremanje (upload) dokaza o uplati u PDF formatu. 2. Unos digitalnog potpisa putem "Signature Pad" komponente na frontendu. Tek kada se ovi uslovi ispune, status prekršaja se menja iz PENDING u PAID. Dodatno, omogućeno je i generisanje PDF uplatnica za svaki prekršaj direktno iz sistema.





5.3. Evidencija Ukradenih Vozila

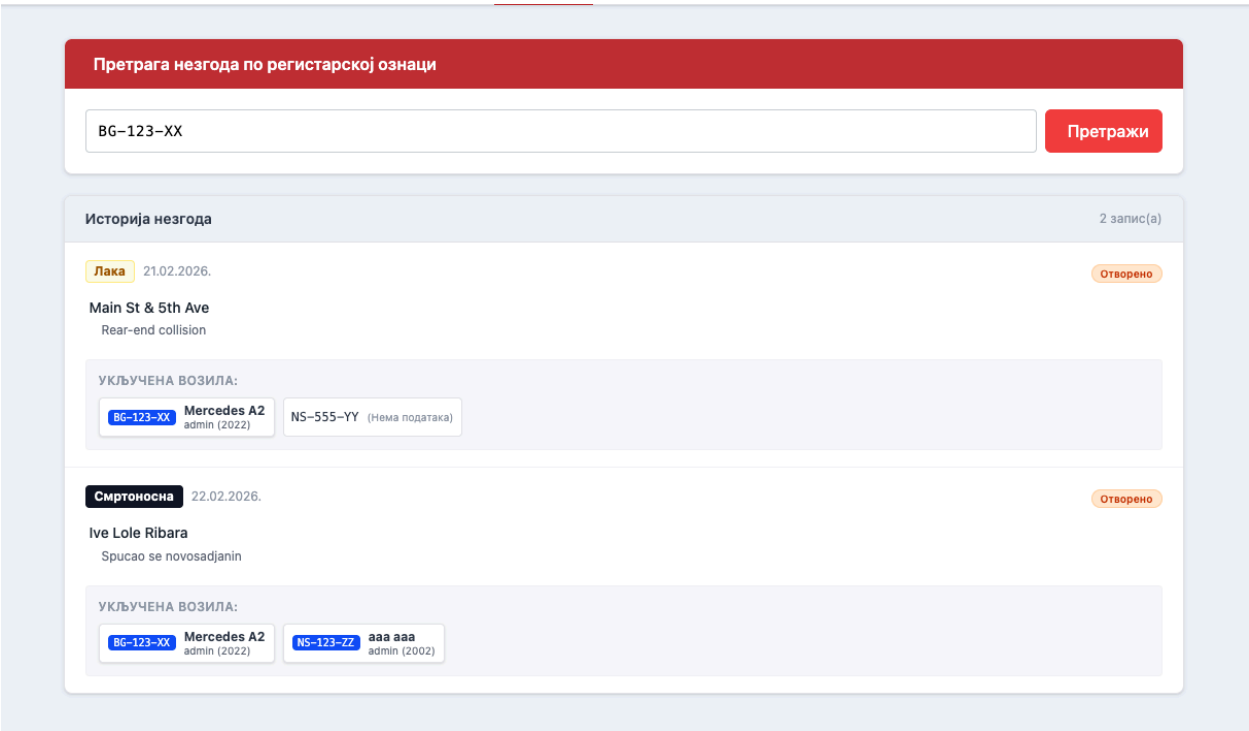
Građani mogu prijaviti krađu svog vozila kroz poseban formular. Prijava se evidentira u *TrafficPoliceService* bazi sa statusom ACTIVE. Ova informacija je od kritične važnosti za integraciju; ukoliko neko pokuša da registruje ovo vozilo na portalu eUprave, *VehicleService* će prilikom provere dobiti informaciju o krađi i automatski odbiti zahtev za registraciju ili prenos vlasništva. Unapređenjem frontenda, prikaz ukradenih vozila sada uključuje dodatne podatke poput marke i modela, pribavljenih asinhronim pozivom ka *VehicleService*-u.



5.4. Evidencija i obrada saobraćajnih nezgoda

Sistem pruža naprednu funkcionalnost za evidentiranje saobraćajnih nezgoda *(Accidents)*, koja obuhvata praćenje svih učesnika incidenta. Prilikom prijave nezgode, MUP službenik unosi detalje uključujući lokaciju, opis, stepen težin (od "Lake" do "Smrtonosne") i registarske oznake svih umešanih vozila.

Na bekendu, *TrafficPoliceService* primenjuje dvostepeni proces obrade. Prvo se podaci garantovano upisuju u lokalnu SQL Server bazu, pri čemu se oznake svih umešanih vozila čuvaju u CSV *(Comma-Separated Values)* formatu (npr. `BG-123-AA, NS-456-BB`).Prikaz nezgoda na klijentskoj strani je visoko optimizovan. Kada građanin ili službenik pretražuje istoriju nezgoda po jednoj registarskoj oznaci, sistem koristi `LIKE` upit na bazi da pronađe sve nezgode gde se ta oznaka nalazi unutar CSV stringa. Nakon što frontend dobije listu nezgoda, on dinamički parsira niz umešanih tablica i upućuje asinhrone zahteve ka *VehicleService-u* (putem *Go proksija*) kako bi dobavio detaljne metapodatke (marka model) za svako vozilo koje je učestvovalo u nezgodi.



6. Zaključak i Dalja Unapređenja

Razvoj *TrafficPolice* mikroservisa uspešno zaokružuje funkcionalnosti simuliranog sistema eUprave, demonstrirajući pun potencijal labavo spojenih (loosely coupled) sistema. Odvojena arhitektura, gde ASP.NET Core komunicira sa Golang servisom putem HTTP-a, pokazuje kako različiti timovi mogu koristiti tehnologije koje im najviše odgovaraju, a da krajnji korisnik ima jedinstveno iskustvo kroz jedan frontend portal. Sistem uspešno zadovoljava sve zadate arhitekturne uslove i funkcionalnosti.

Moguća dalja unapređenja uključuju: **1. Asinhrona razmena poruka (Message Queues):** Umesto direktnih HTTP poziva (poput obaveštavanja NotificationService-a), uvođenje *RabbitMQ* ili *Kafka* sistema bi obezbedilo da se poruke nikada ne izgube, čak i ako je ciljni servis privremeno nedostupan. 2. **Integracija Payment Gateway-a:** Trenutno građani sami prilažu PDF dokaz o uplati. Uvođenjem direktne integracije sa servisima poput Stripe-a ili nacionalnog e-plati sistema, verifikacija bi bila potpuno automatska. 3. **Grafička Analitika:** Implementacija analitičkih tabli (Dashboards) na Admin panelu koji bi MUP-u prikazivali statistiku o vrstama prekršaja po lokacijama i vremenskim periodima, pomažući u prevenciji.

Razvoj ovog projekta pružio je praktično iskustvo u primeni mikroservisne arhitekture, Go programskog jezika u sinergiji sa modernim Angular frontendom, kao i izazovima među-servisne komunikacije.

7. Reference

[1] Golang zvanična dokumentacija, https://go.dev/doc/

[2] Gin Web Framework, https://gin-gonic.com/

[3] GORM - The fantastic ORM library for Golang, https://gorm.io/

[4] Microservices architecture, Martin Fowler

[5] GOV.UK, How Hackney Council built an API Platform.

[6] PrimeNG - UI Components for Angular, https://primeng.org/

[7] Entity Framework Core Documentation

[8] MailTrap Documentation

[9] Microsoft SQL Server Documentation

[10] Angular Documentation, https://angular.dev/

[11] Tailwind CSS Documentation, https://tailwindcss.com/

[12] TypeScript Documentation

[13] Docker Documentation